

VideoCAD: A Large-Scale Video Dataset for Learning UI Interactions and 3D Reasoning from CAD Software

Brandon Man* **Ghadi Nehme*** **Md Ferdous Alam** **Faez Ahmed**
 Department of Mechanical Engineering, Massachusetts Institute of Technology
 Cambridge, MA 02139, USA
 {bm557, ghadi, mfallam, faez}@mit.edu

Abstract

Computer-Aided Design (CAD) is a time-consuming and complex process, requiring precise, long-horizon user interactions with intricate 3D interfaces. While recent advances in AI-driven user interface (UI) agents show promise, most existing datasets and methods focus on short, low-complexity tasks in mobile or web applications, failing to capture the demands of professional engineering tools. In this work, we introduce VideoCAD, the first attempt at engineering UI interaction learning for precision tasks. Specifically, VIDEOCAD is a large-scale synthetic dataset consisting of over 41K annotated video recordings of CAD operations, generated using an automated framework for collecting high-fidelity UI action data from human-made CAD designs. Compared to existing datasets, VIDEOCAD offers an order of magnitude higher complexity in UI interaction learning for real-world engineering tasks, having up to a $20\times$ longer time horizon than other datasets. We show two important downstream applications of VIDEOCAD: learning UI interactions from professional precision 3D CAD tools and a visual question-answering (VQA) benchmark designed to evaluate multimodal large language models' (LLM) spatial reasoning and video understanding abilities. To learn the UI interactions, we propose VIDEOCADFORMER - a state-of-the-art model in learning CAD interactions directly from video, which outperforms multiple behavior cloning baselines. Both VIDEOCADFORMER and the VQA benchmark derived from VIDEOCAD reveal key challenges in the current state of video-based UI understanding, including the need for precise action grounding, multi-modal and spatial reasoning, and long-horizon dependencies. Dataset and code available at: <https://github.com/BrandonMan123/VideoCAD>

1 Introduction

Professional engineering design software, such as Computer-Aided Design (CAD) tools, presents significant challenges for human users due to their complex user interfaces, intricate workflows, and high-precision requirements [1]. Unlike typical consumer applications, where tasks are completed through simple UI interactions, CAD operations require structured, multi-step processes involving 3D spatial reasoning and parametric modeling. Becoming proficient in these tools can take years of expertise, and it is often very difficult to automate. Although AI systems have the potential to automate a large portion of such critical workflow for a significant productivity boost, current AI-driven automation methods struggle to generalize due to a lack of large-scale, annotated datasets for learning from CAD interfaces.

Existing research in user interface navigation has largely focused on web and mobile applications, where task completion involves relatively short time horizons and does not require any 3D reasoning

*Equal contribution.

capabilities. Benchmarks such as MiniWob++ [2] and RICO [3] provide valuable insights into learning from UI interactions, but they lack the complexity of CAD environments, where tasks involve nested dependencies, precise geometric constraints, and tool-based interactions beyond simple button clicks. Furthermore, while behavior cloning from videos has shown promise in robotic manipulation and gaming environments, its application to software UI understanding remains underexplored due to the scarcity of high-quality, video-annotated UI datasets.

To bridge this gap, we introduce VIDEOCAD, a large-scale dataset comprising over 41K synthetic video demonstrations of CAD modeling tasks and containing fine-grained action annotations. To generate VideoCAD, we leverage a public dataset of parametric CAD models (DeepCAD [4]) made by humans using Onshape [5] (see Appendix A), a widely-used browser-based professional CAD platform. We map the construction sequences of CAD models into UI actions performed within OnShape.

We demonstrate the effectiveness of our dataset, we provide two important downstream applications of VIDEOCAD. First, we develop a novel transformer-based framework, VIDEOCADFORMER, that outperforms existing state-of-the-art methods in learning from such rich user interactions for CAD designs. We benchmark our model against other state-of-the-art behavior cloning baselines and show that VIDEOCADFORMER is able to reason about long-horizon UI interaction tasks that involve 3D reasoning better than other models by as much as 20%. Second, we also introduce VIDEOCADQA, a synthetically generated multiple-choice VQA dataset derived from VIDEOCAD that includes questions that evaluate 3D reasoning and video understanding capabilities of LLMs. Our experiments highlight a critical gap in spatial reasoning of LLMs in precise engineering tasks.

VIDEOCAD has the potential to serve as a valuable benchmark for advancing research in AI-driven UI navigation, software automation, and CAD generation. By providing an open-source dataset with rich annotations, we aim to facilitate breakthroughs in learning from software demonstrations and bridging the gap between computer vision, reinforcement learning, and CAD modeling. More specifically, our contributions can be summarized as follows:

- **A large-scale, high-fidelity CAD interaction dataset:** VIDEOCAD consists of over 41K annotated videos, capturing diverse CAD workflows with both low-level UI actions and high-level modeling operations.
- **Behavior cloning benchmarks and a state-of-the-art model:** We evaluate multiple baselines on VIDEOCAD and propose VIDEOCADFORMER, a transformer-based architecture that achieves state-of-the-art performance on long-horizon CAD action prediction.
- **A VQA benchmark for 3D and spatiotemporal reasoning:** We present a case study of extending VideoCAD into a VQA benchmark, introducing a suite of 1,200 visual questions that probe LLMs’ ability to perform fine-grained 3D reasoning and temporal understanding over CAD videos.

2 Related works

User Interface Navigation. Automating user interface navigation has been a long-standing challenge in AI research. Works such as MiniWob++ [6] and Android In The Wild (AITW) [7] introduced datasets for web and mobile UI interaction, allowing agents to learn structured policies for performing simple tasks. However, these datasets focus on relatively simple tasks with short time horizons. More recent works such as WebShop [8] have extended these capabilities to complex real-world scenarios where an agent must reason over long sequences of actions to achieve its goal.

UI Agents and Behavior Cloning from Videos. Learning from demonstrations has been widely applied to UI interaction. Behavioral cloning, where an agent learns from human demonstrations, has been successfully applied in robotic manipulation [9] and game environments [10]. However, its application to GUI-based tasks is still in its early stages. Recent studies, such as ActionBert [11] and AssistGUIs [12], explored using large-scale datasets for predicting UI interactions from multimodal inputs. Our work extends these efforts by introducing a dataset specifically tailored for CAD interactions, where the action space is significantly more complex.

CAD Datasets for Learning-Based Modeling. Parametric CAD datasets remain scarce compared to mesh or point cloud collections. The ABC dataset offers 1 million B-rep models with precise geometry but lacks procedural data [13]. DeepCAD supplements this by providing 178K models with

full construction histories, enabling sequence-based generative modeling [4]. Fusion 360 Gallery contributes 8.6K human-authored CAD programs, capturing sketch-extrude workflows and assembly hierarchies [14]. MFCAD and MFCAD++ focus on machining feature recognition, supplying labeled B-rep models for supervised learning [15, 16]. However, there is a lack of datasets that can capture the fine-grained UI interactions or the visual and geometric reasoning needed to model 3D CAD workflows. VIDEOCAD addresses this gap by providing video demonstrations of CAD construction, offering temporally grounded UI state changes and visual context to support multimodal learning of design behaviors.

AI-Driven CAD Generation. Generative modeling of CAD has been an active research area in 3D vision [17, 18, 19, 20]. DeepCAD [4] introduced one of the first deep learning models capable of generating parametric CAD sequences, while subsequent works such as SketchGraphs [21] have provided datasets for learning structured CAD designs. Recent advances in generative AI, such as transformer-based models for sketch generation [22], conditional latent diffusion model for CAD sequence generation from images [23] and direct CAD construction [24], highlight the potential for learning CAD from visual demonstrations. VIDEOCAD complements these efforts by providing an extensive dataset of video demonstrations, enabling new paradigms in learning CAD construction from human-like interactions.

3 VIDEOCAD Dataset

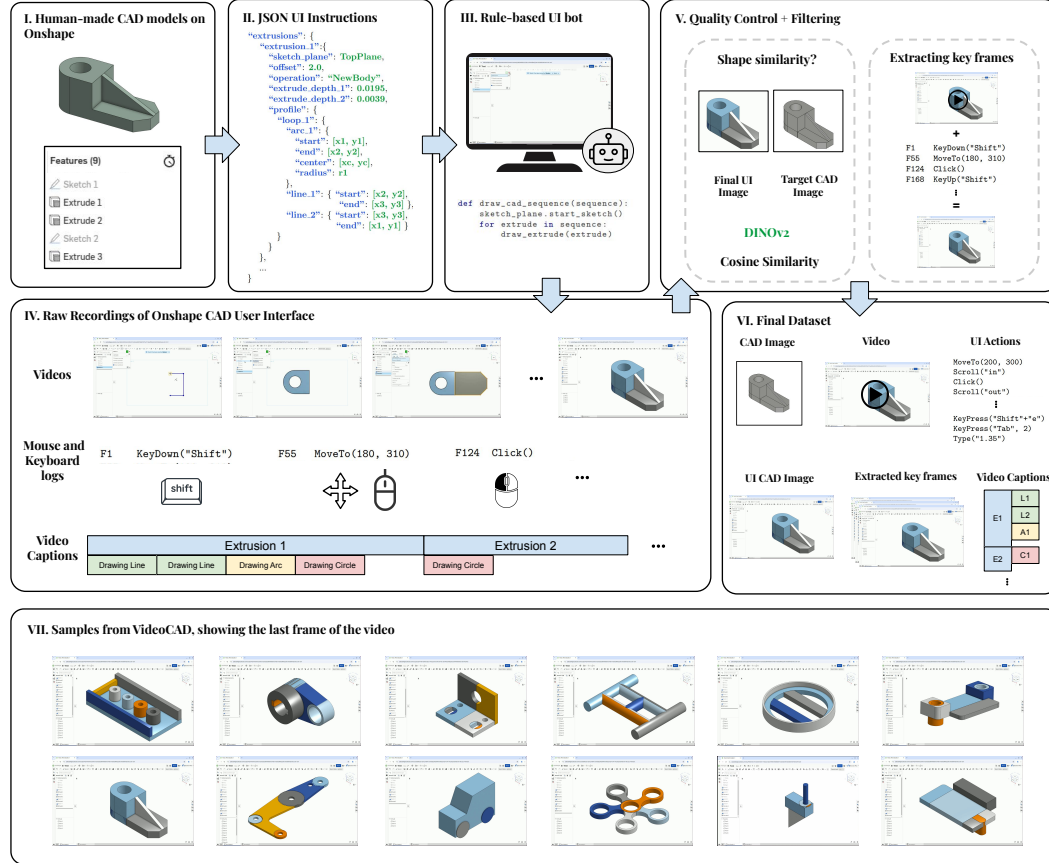


Figure 1: Illustration of the VIDEOCAD dataset pipeline: human-authored CAD sequences are converted into UI instructions and executed via a rule-based bot to record videos. Quality filtering (DINOv2), keyframe extraction, and action alignment produce structured video-action pairs.

VIDEOCAD is a large-scale dataset $\mathcal{D}$ comprising 41,005 synthetic videos of 3D CAD model construction, generated from human-authored designs. Each data sample consists of a multiframe video of an automated agent interacting with the OnShape CAD interface [5], a free, browser-based

CAD platform, accompanied by two levels of timestamped action annotations and a ground-truth target rendering in isometric view. The annotations consist of low-level actions and high-level actions. Low-level actions capture UI interactions such as clicking, typing, and mouse movements, with timestamps marking when each action occurred. These low-level actions correspond to the action space discussed in section 4. High-level actions align with CAD modeling operations, recording when primitives such as extrusions and loops are constructed. High-level actions correspond to the CAD primitives found in DeepCAD. More specifically, VIDEOCAD consists of the following, $\mathcal{D} = \{(\mathbf{X}_i, \mathbf{I}_i, \mathbf{a}_i)\}_{i=1}^N$ where $\mathbf{X}_i \in \mathbb{R}^{t \times H \times W \times C}$ is the video frame, $\mathbf{I}_i \in \mathbb{R}^{H' \times W' \times C'}$ is the corresponding image of the target shape, and $\mathbf{a}_i \in \mathbb{R}^{t \times d}$ is the corresponding action vector that represents the UI actions taken to create the CAD with the desired shape.

3.1 Dataset Generation Pipeline

Sketch-extrude CAD. To create the VIDEOCAD dataset, we leverage the DeepCAD sequence representation [4], which models CAD construction as a sequence of sketches and extrusions. Each 2D sketch consists of one or more closed loops formed by geometric primitives—lines, arcs, and circles—parameterized by their spatial properties (e.g., start/end points, midpoints, radii). These sketches are extruded using parameters such as orientation (Euler angles θ, ϕ, γ), 3D offsets (p_x, p_y, p_z), scale s , bidirectional depths (e_1, e_2), and Boolean operations β (e.g., join, cut). This design framework is visualized in Figure 2.

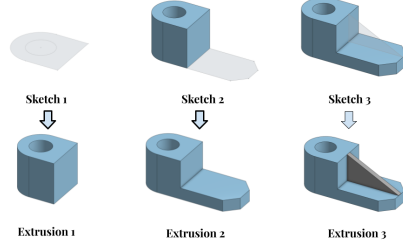


Figure 2: An example of a step-by-step construction of a multi-extrusion CAD model using sketch and extrusion operations.

CAD uses a sequence of parametric commands to create the shape step by step from scratch. Although there are a lot of CAD operations in existing commercial platforms, the two most widely used CAD operations are sketch and extrusion [4]. The purpose of the sketch command is to create the 2D sketch on a plane and then the extrusion command extrudes the sketch to create a 3D shape. To create complex and realistic 3D shapes, multiple sketches on different planes and multiple extrusion operations are required.

From Human-Created CAD Models to UI instructions. We systematically convert these sequences of human-made CAD models into executable sequences of UI actions compatible with Onshape’s interface. To automate CAD modeling within Onshape, we developed a hybrid UI interaction framework driven by a rule-based bot. Each CAD model is built incrementally by defining sketch planes, either default or custom offset planes, followed by drawing geometric primitives (lines, arcs, circles), and performing extrusion operations with specified parameters. Further details on the translation from DeepCAD sequences to CAD UI actions are described in Appendix ??.

Rule-Based Bot Execution. We use a hybrid rule-based bot to execute the instruction sequences within Onshape. The bot uses: **Selenium** for DOM-level browser automation, such as opening sketch menus, toggling extrusion modes, or navigating dialog boxes. **PyAutoGUI** for low-level pixel-based interactions within the sketch canvas, including drawing curves and typing parameter values. Since Onshape does not expose a public sketching API, this low-level simulation is essential.

All (x, y) interactions are normalized to the $(0, 1)$ screen space to ensure scale invariance. A timeout-based retry mechanism (3 retries, 5s each) is used for all UI actions. Episodes are terminated after three consecutive failures—most frequently caused by interface lags or rendering issues. During execution, we record high-resolution video at 60 FPS and log every UI action with sub-second alignment, deploying the system across 64 cloud VMs. In total, our bot successfully reconstructed 44,292 CAD models out of 71,424 attempts (62.0%).

Screen Recording and Logging. While the bot executes, we record high-resolution screen captures at 60 FPS using a custom Python video recorder. Simultaneously, we log every executed UI action (mouse event, keyboard press, selection change), each tagged with its precise frame index. This produces action–video alignment at sub-second granularity. The system is deployed across 64 Google

Cloud VMs using ‘Xvfb’ to simulate GUI displays. Videos and logs are streamed to Google Cloud Storage. In total, over 118 days of screen recordings were generated in under one week.

Introducing Human-like Heuristics to VideoCAD To balance realism and learnability in the presence of long-horizon 3D reasoning tasks (see Section 3.3), we introduce a few lightweight human-inspired heuristics into VideoCAD [25]. First, we add randomized delays between actions, with the bot waiting 0.2–0.5 seconds before interacting—mimicking human hesitation. Additionally, surface selections during sketching are made by randomly sampling a point on the surface, rather than always selecting the center. Finally, to emulate the need for precision, the bot performs zoom actions when small features are hard to select, replicating how humans zoom in to refine their input.

Quality Control and Filtering. To ensure high-quality reconstruction, we render the final CAD model from an isometric view and compare it with the human-authored reference using DINOv2 vision embeddings. A cosine similarity threshold is applied to automatically discard inaccurate reproductions. We manually verified that this metric correlates well with geometric correctness (see Appendix C). After filtering, keyframes are extracted for each UI action based on the logged frame index, yielding a sequence of temporally aligned image–action pairs.

3.2 Dataset Composition and Statistics

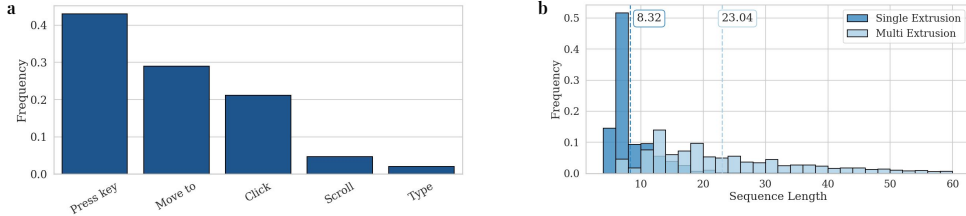


Figure 3: Statistical distributions of CAD UI actions and sequence lengths. **a.** Action command frequencies. **b.** Sequence length frequencies.

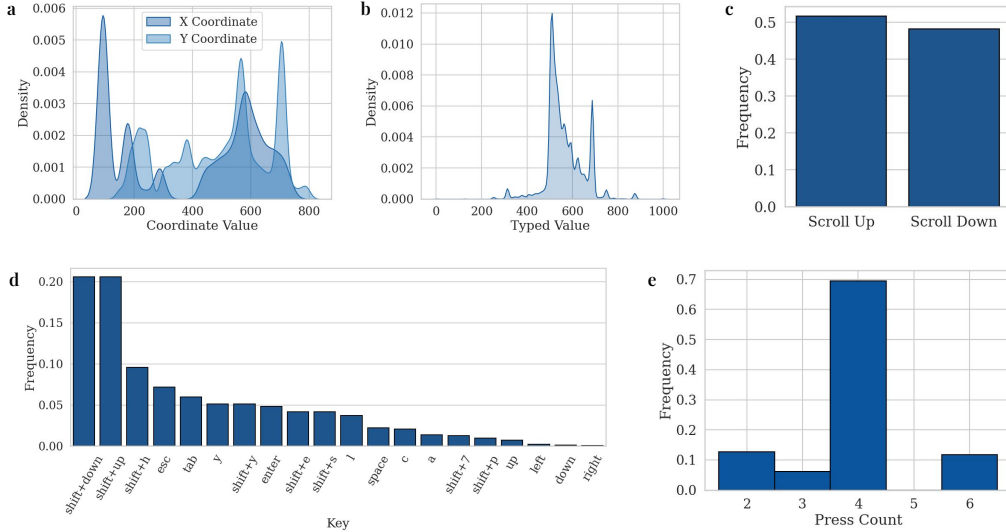


Figure 4: **(a)** Overlaid kernel density estimates for the X- and Y-coordinate distributions of mouse movements. **(b)** Kernel density of numeric values entered via the ‘Type’ action. **(c)** Relative frequencies of scroll directions (up vs. down). **(d)** Frequency of individual key presses, sorted from most to least common. **(e)** Histogram of the number of times the ‘tab’ key is pressed.

We focus exclusively on multi-extrusion sequences from the DeepCAD dataset to create VIDEOCAD. Multi-extrusion sequences involve substantially longer action sequences (Figure 3) and multi-surface operations—making them more challenging for learning-based models. Each episode includes: 1) A rendered target image of shape $3 \times 224 \times 224$, in isometric view, 2) A full-resolution video at 1600×1000 , recorded at 60 FPS, 3) A sequence of action tuples aligned with the video timeline. Figure 3 shows the action commands distribution and figure 4 shows the distribution of values of each parameter.

3.3 Benchmarking

Engineering UI is often more complex than traditional UI due to the necessity of many operations for precision control of the user. This can be seen in CAD software due to the complexity of the UI needed for precise spatial reasoning. To show this complexity, we benchmark VIDEOCAD extensively against other existing UI-agent datasets from literature in Table 1 (Appendix D).

Environment	# Samples	Time Horizon	3D Reasoning	Precise Elements	Avg. # Elements
OSWorld	369	15*	✗	✓	—
Mind2Web	2,350	7.3	✗	✗	1,135
WebArena	812	—	✗	✗	—
VisualWebArena	910	35*	✗	✗	—
TheAgentCompany	175	40	✗	✗	—
WorkArena	33	15	✗	✓	—
WebShop	12,000	11.3	✗	✗	38
OmniAct	9,802	—	✗	✓	—
WebLinx	2,337	43	✗	✗	1,849
AITW	715,142	6.5	✗	✗	—
MMINA	1,050	12.9	✗	✓	601
MetaGUI	1,125	—	✗	✗	79
PixelHelp	187	4.2	✗	✗	—
AndroidWorld	116	18.1	✗	✓	—
AgentStudio	304	30*	✗	✓	—
MoTIF	4,707	4.4	✗	✗	188
AndroidArena	116	11.4	✗	✗	—
WindowsAgentArena	154	8.1	✗	✓	—
MiniWoB++	125	3.6	✓	✗	28
GUI-WORLD	12,379	10.97	✓	✓	—
VideoCAD	41,005	186	✓	✓	6,740

Table 1: Full Comparison of multi-environment benchmarks for GUI interaction.

* The max is used instead of the average as the average is not reported

We compare dataset complexity across five metrics: **# Samples** – total number of samples; **Time Horizon** – number of UI actions per task; **Requires 3D Reasoning** – whether tasks involve manipulating 3D coordinates; **Precise Element** – whether agents must act via xy coordinates (e.g., canvases) rather than DOM selectors, requiring spatial and visual reasoning; and **Average UI Elements** – mean number of HTML elements per interface (reported only for datasets with HTML tree access).

As shown in Table 1, VIDEOCAD stands out across multiple dimensions. It features the longest time horizon— $4\times$ that of the next closest dataset (WebLinx)—and is the second-largest dataset after Android in the Wild, with over $50\times$ more samples than the median (812). It is one of only two datasets requiring 3D spatial reasoning and pixel-level xy interactions, challenging agents to operate beyond text-based commands. For datasets with DOM access, VIDEOCAD also contains $6\times$ more UI elements than the average web page in Mind2Web.

4 VideoCADFormer: An Autoregressive Transformer to Predict CAD Actions

Commercial CAD Software as an Environment. We model CAD construction as a sequential decision process, where an agent observes UI frames and predicts low-level actions to recreate a target 3D model. Each trajectory in VIDEOCAD is a sequence $\tau = \{(\mathbf{I}, \mathbf{o}_t, \mathbf{a}_t)\}_{t=0}^T$, where $\mathbf{I}$ is a fixed isometric view of the target shape, $\mathbf{o}_t$ is the UI frame at timestep t , and $\mathbf{a}_t$ is the expert action. The agent learns a policy $\pi_\theta(\mathbf{a}_t \mid \mathbf{o}_{t-k:t}, \mathbf{I})$ via behavior cloning, trained to minimize a supervised loss

over expert demonstrations $\mathcal{L}_{BC} = \mathbb{E}_{\tau \sim \mathcal{D}} \sum_t \ell(\pi_\theta(\mathbf{o}_{t-k:t}, \mathbf{I}), \mathbf{a}_t)$, where ℓ is a structured prediction loss over commands and parameters. Our setup can be extended to reinforcement learning with rewards based on geometric similarity (see Appendix F for Chamfer Distance reward details).

Observation Space. At each timestep t , the model observes a grayscale UI image $\mathbf{o}_t \in \mathbb{R}^{224 \times 224 \times 1}$ representing the current design state, and a target CAD image $\mathbf{I} \in \mathbb{R}^{224 \times 224 \times 1}$, which remains fixed throughout the episode. Together, these visual inputs provide both local progress and global goal context, enabling the model to ground its predictions in both current and intended geometry.

Action Space. To interact with the CAD interface, the agent issues low-level UI commands. Each action $\mathbf{a}_t$ is a structured tuple: $\mathbf{a}_t = (c_t, p_t^1, \dots, p_t^{d_t})$, c_t is the command type index, d_t is the number of parameters for command c_t , p_t^i is the i -th parameter for command c_t . Each action is a fixed-length vector: $\mathbf{a}_t = (c_t, x_t, y_t, k_t, n_t, s_t, v_t)$, where each field corresponds to the parameter(s) used by one or more commands. Table 2 is the set of low-level UI commands and their associated parameters. Unused fields are padded with -1 . All parameter values are discretized into 1000 classes, enabling the action prediction task to be cast as a multi-class classification problem. At the end of each CAD video, we set the part in isometric view; this action serves as our end-of-sequence command.

Command	Description	Parameters
MoveTo	Move pointer to screen coordinate	x_t, y_t : pointer location
PressKey	Press keyboard key(s)	k_t : key index, n_t : press count
Scroll	Scroll to zoom or pan	s_t : scroll amount
Type	Enter numerical value	v_t : typed value
Click	Left mouse click	$\emptyset$

Table 2: Structured action representation in VIDEOCAD. Each command type is mapped to a consistent 7D vector used for classification. Unused fields are set to -1 .

4.1 Model Architecture

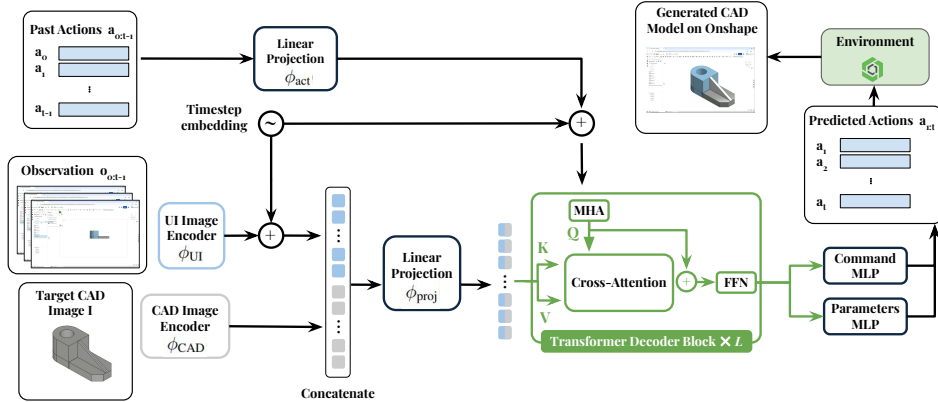


Figure 5: Overview of VIDEOCADFORMER for CAD UI action prediction. The model encodes the target image and past UI frames via ViT, fuses them with projected past actions using a cross-attention decoder, and predicts the next action to iteratively build the CAD model in Onshape.

We propose VIDEOCADFORMER, an autoregressive transformer tailored for CAD UI modeling (Figure 5). The model predicts low-level actions conditioned on a target CAD image, past UI-rendered frames, and prior actions. Visual inputs are processed by separate ViT encoders and fused via linear projection with timestep embeddings, capturing static goals and canvas evolution. Action tokens are embedded and passed to a causal transformer decoder with two key inductive biases: a causal mask to enforce autoregressive prediction, and a window mask to focus attention on recent context—reflecting the short-term dependency of UI commands. The decoder outputs command

and parameter logits through two heads, with command-dependent masking to suppress irrelevant parameters. Full architectural details, training details and ablation studies are in Appendix E.

Input Representation. At each timestep t , the model receives: a target CAD image $I \in \mathbb{R}^{H \times W \times 1}$, a sequence of past UI-rendered frames $o_{t-k:t-1} \in \mathbb{R}^{k \times H \times W \times 1}$, a sequence of previous actions $a_{0:t-1} \in \mathbb{R}^{k \times d}$, and timestep embeddings $\phi_{\text{time}}(t) \in \mathbb{R}^h$.

Visual Encoding. Each input image (target CAD and UI-rendered frames) is processed through a ViT encoder followed by a linear projection into a hidden dimension h . We define:

$$v^I = \phi_{\text{CAD}}(I) \in \mathbb{R}^h, \quad v_t^o = \phi_{\text{UI}}(o_t) + \phi_{\text{time}}(t) \in \mathbb{R}^h$$

The sequence of UI frame embeddings is concatenated with the static CAD embedding and projected:

$$z_t^{\text{image}} = \phi_{\text{proj}}([v^I; v_t^o]) \in \mathbb{R}^h.$$

Action and Timestep Embeddings. Each previous action $a_\tau \in \mathbb{R}^d$ is projected to the hidden space:

$$z_\tau^{\text{act}} = \phi_{\text{act}}(a_\tau) + \phi_{\text{time}}(\tau), \quad z_\tau^{\text{act}} \in \mathbb{R}^h.$$

All inputs are activated using tanh and passed to the transformer decoder.

Transformer Decoder. We use an L -layer causal transformer decoder with hidden size h and n attention heads. The decoder operates over a sequence of action embeddings $Z^{\text{act}} \in \mathbb{R}^{T \times h}$ (target) and visual memory $Z^{\text{image}} \in \mathbb{R}^{T \times h}$ (source):

$$H_t = \text{TransformerDecoder}(Z^{\text{act}}, Z^{\text{image}}, \mathbf{M}^{\text{causal}}, \mathbf{M}^{\text{window}}).$$

We use two attention masks: a **causal mask** $\mathbf{M}^{\text{causal}} \in \mathbb{R}^{T \times T}$ is an upper-triangular matrix with $-\infty$ in positions where future tokens should be masked and 0 elsewhere

Action Prediction. The hidden states $H_t \in \mathbb{R}^{T \times h}$ are decoded into actions using two heads:

$$\hat{c}_t = \text{softmax}(W_c H_t + b_c), \quad \hat{c}_t \in \mathbb{R}^5, \quad \hat{p}_t = \text{softmax}(W_p H_t + b_p), \quad \hat{p}_t \in \mathbb{R}^{6 \times 1000}.$$

Command-dependent masks $M_{\hat{c}_t} \in \{0, 1\}^6$ are applied to suppress unused parameter outputs. Invalid values are set to -1 .

4.2 Evaluation Metrics

Command and Parameter Accuracy. We report the classification accuracy of predicted commands $\hat{c}_t$ and parameters $\hat{p}_t$ across the entire test set. Command accuracy measures the fraction of correctly predicted command types. Parameter accuracy is computed per action, conditioned on correct command prediction.

$$\mu_{\text{cmd}} = \frac{1}{T} \sum_{t=1}^T \mathbb{1}[\hat{c}_t = c_t], \quad \mu_{\text{param}} = \frac{1}{T} \sum_{t=1}^T \frac{1}{d_t} \sum_{i=1}^{d_t} \mathbb{1}[\hat{p}_t^{(i)} = p_t^{(i)}] \cdot \mathbb{1}[\hat{c}_t = c_t]$$

Offline Closed-Loop Execution Performance. We evaluate the stability of models under full-sequence autoregressive rollouts by computing the percentage of perfectly predicted actions—defined as exact matches with the ground truth—across all sequences in the test set. We report the mean, minimum, and maximum values of this percentage per method and breakdowns by sequence length. Sequences are categorized into short (0–120), medium (120–200), and long (200+) bins based on test set percentiles.

Geometric Fidelity. We evaluate geometric accuracy by executing model-predicted actions in Onshape and rendering the resulting CAD models. On 100 test sequences, we compute mean bidirectional Chamfer Distance (CD) after PCA alignment (Appendix F). A sequence is considered successful if $\text{CD} < 0.02$, a threshold chosen empirically based on human evaluation, where shapes below this value are visually indistinguishable (Appendix F). Invalid sequences (i.e., those failing to mesh) are also reported.

5 Results

Accuracy and Perfect Sequences. We use Video Pre-training (VPT) [10] (a leading method in offline behavior cloning for Minecraft), Pix2Act [26] and Pearce et. al. [27] as baselines. As shown in Table 7, VIDEOCADFORMER achieves the highest command and parameter accuracy across all evaluated models, outperforming VPT, Pix2Act, and Pearce et al. It also leads across all metrics of perfect action sequence prediction. The model shows consistent improvements over VPT across all levels of task difficulty. These results highlight VIDEOCADFORMER’s robustness in executing complete CAD sequences without error, particularly in long-horizon settings where small mistakes compound. The increasing performance gap with task difficulty indicates better generalization under growing complexity.

Geometric Fidelity. Table 4 evaluates the final CAD quality by executing the predicted sequences in Onshape. VIDEOCADFORMER outperforms VPT in both short and long sequences in terms of Chamfer-based success rate, while VPT performs slightly better on medium sequences. Overall, VIDEOCADFORMER achieves lower Chamfer distance and a lower proportion of invalid CAD models. These results suggest that stronger sequence-level prediction directly translates to more accurate 3D model reconstruction.

Table 3: Evaluation metrics across task difficulty levels.

	μ_{cmd} (%)	μ_{param} (%)	Perfectly Predicted Actions (%)					
			Mean	Max	Min	Short	Medium	Long
Pix2Act [26]	20.44	2.61	2.84	22.03	0.00	2.28	2.63	3.60
Pearce et al. [27]	42.60	0.55	0.68	10.80	0.00	0.83	0.69	0.51
VPT [10]	96.25	78.72	83.81	100.00	43.59	88.51	82.77	80.12
VIDEOCADFORMER	98.08	82.35	87.54	100.00	65.67	90.08	87.08	85.46

Table 4: Performance by sequence length on Chamfer success rate ($<0.02 \uparrow$), mean Chamfer distance ($\downarrow$), and invalid sample rate ($\downarrow$). Overall metrics are averaged across categories.

Method	Success Rate (%) $\uparrow$			Mean	Mean CD $\downarrow$			Mean	Invalid (%) $\downarrow$
	Short	Medium	Long		Short	Medium	Long		
Human Expert	85.0	96.7	82.8	88.2	0.0097	0.0067	0.0112	0.0092	0.0
Random	2.5	0.0	0.0	0.8	0.1038	0.1075	0.0972	0.1028	–
VPT	37.5	40.0	3.4	27.0	0.0342	0.0229	0.0882	0.0484	32.3
VIDEOCADFORMER	55.0	33.3	24.1	37.5	0.0273	0.0325	0.0572	0.0390	25.3

5.1 Goal-Driven CAD Generation and Autocompletion

Shape Generation from Scratch. Conditioned only on a target CAD image, our model can generate full action sequences that construct the corresponding geometry from an empty canvas. As shown in Figure 6, the predicted low-level UI actions reliably recreate complex multi-step CAD models. This demonstrates the model’s capacity to plan and execute long-horizon sequences that align with spatial goals purely from visual context.

CAD Model Autocompletion. Beyond generation from scratch, our model can also autocomplete a partially built CAD model when conditioned on both the current intermediate state and a target final image. Given a prefix of UI actions (e.g., an initial sketch and extrusion), the model predicts the remaining sequence needed to match the final design. As shown in Figure 6, this enables intelligent continuation of in-progress designs and assistive AI behavior in iterative modeling workflows.

Failure Cases and Limitations. Model failures are primarily due to inaccurate (x, y) predictions that cause open or slightly distorted sketch loops, preventing valid extrusions. Misclassification between lines and arcs also occurs, especially when curvature is ambiguous. These errors (Figure 11) are often minor and correctable, but they highlight the limitations of image-only supervision and suggest the need for topological constraints or interactive fine-tuning. More details in Appendix H.

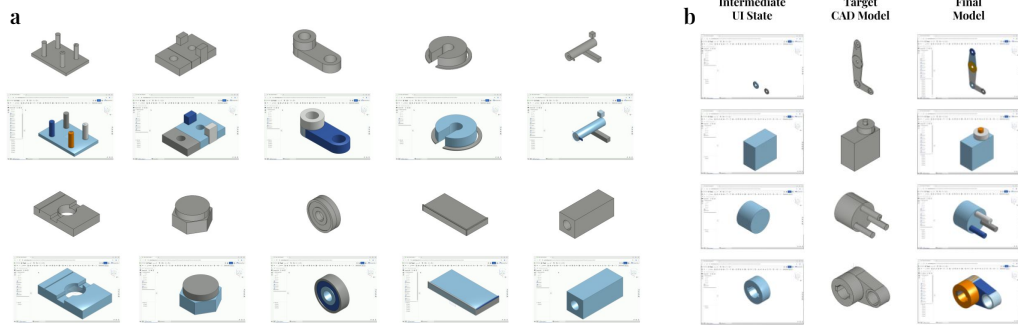


Figure 6: Predicted CAD models from VIDEOCADFORMER, conditioned on (a) a target image for generation from scratch, and (b) a partial UI state for autocompletion.

5.2 Case Study: Evaluating LLMs on 3D Reasoning and CAD Understanding

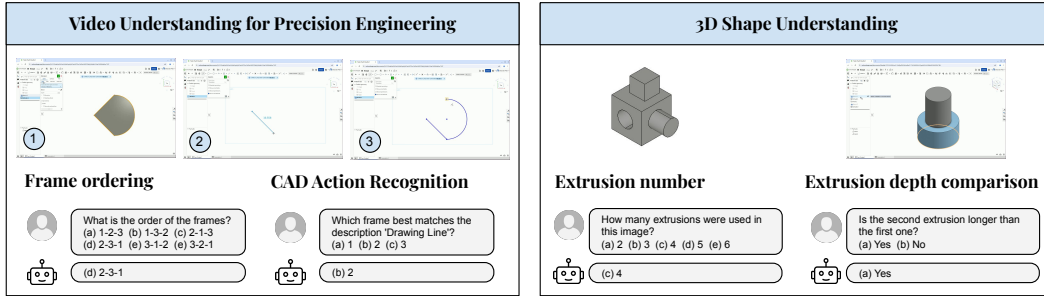


Figure 7: Example questions from the VIDEOCAD VQA benchmark.

LLMs Struggle with 3D Reasoning and CAD Video Understanding. To evaluate the limits of modern multimodal LLMs in spatially grounded engineering domains, we construct a VQA benchmark, VIDEOCAD VQA, which is derived from VIDEOCAD. This benchmark contains multiple-choice multimodal questions that probe fine-grained understanding of both temporal video sequences and 3D geometric properties—tasks essential for CAD modeling. It includes four task types: action recognition, frame ordering, extrusion counting, and depth comparison. Questions are generated automatically from ground-truth UI logs and CAD geometry (Appendix I).

Evaluation Type	gpt-4.1 [28]	claude-3-7 [29]	qwen2.5-vl [30]	o3 [31]	gemini-2.5 [32]	random
Extrusion Shape Prediction	27.0	22.5	19.5	22.5	25.5	29.0
Number of Extrusion Estimation	47.0	37.5	47.0	45.0	38.0	21.4
Extrusion Difference Prediction	73.5	80.0	62.0	71.5	71.0	49.1
Sketch Ordering	37.5	29.5	41.0	37.0	61.0	35.0
Sketch Identification	62.0	48.5	43.5	48.5	59.0	21.5
Plane Identification	87.0	86.5	86.0	91.5	89.5	34.0
CAD Primitive Identification	84.1	80.1	78.6	75.1	82.6	40.3
Sequence Prediction	81.0	68.0	70.5	77.5	79.2	36.5
Video Frame Sequencing	36.0	23.0	32.5	80.0	73.2	17.4
Hole Detection	92.0	53.5	79.5	88.4	81.0	50.0
Symmetry Detection	18.5	19.0	12.0	27.9	27.0	12.5

Table 5: Performance of various vision-language models on the CAD-VQA benchmark across 11 evaluation tasks. Metrics are accuracy (%).

We perform 0-shot evaluation on VIDEOCAD VQA by feeding a template followed by the prompt to each LLM. If there is no valid answer in the model’s response, we perform random selection as a remedy. We evaluate the model 3 times for each question. Table ?? shows results across multimodal LLMs. Despite their strong performance on general VQA benchmarks, current models underperform across most categories. For instance, GPT-4.1 achieves only 47% accuracy on extrusion number estimation and 36% on frame ordering, highlighting persistent challenges in grounded visual reasoning and temporal understanding in CAD workflows. These results underscore a critical gap in LLMs’ ability to interpret complex video-based modeling workflows. Importantly, this VQA benchmark serves as a case study showcasing how VIDEOCAD enables rigorous evaluation of spatial and procedural understanding in LLMs. Many additional multimodal tasks—such as symmetry detection, subpart recognition, or parameter estimation—can be derived from the same dataset and are discussed in Appendix I.

LLMs Fail as UI Agents for Precision CAD Tasks. We also benchmark LLMs as UI agents in complex engineering software. Using BrowserGym [33], we prompt models to perform CAD construction tasks within Onshape based only on a target screenshot. Each agent is given 200 steps and must produce *xy* coordinates corresponding to UI actions (e.g., drawing, extrusion). Tasks are drawn from the 10 shortest sequences in VIDEOCAD, with early termination if no meaningful canvas modification occurs. No LLM—including GPT-4.1, Claude-3.7, and Gemini-2.5—is able to successfully complete a full CAD construction. We anticipate that this happens due to the need for pixel-level precision, long-horizon spatial planning, and consistent 3D reasoning capabilities for CAD modeling unlike web tasks (e.g., spreadsheets or browser automation [34]).

6 Limitations and Future Work

While VIDEOCAD offers a high-fidelity benchmark for CAD UI modeling, it has several limitations. All trajectories are synthetically generated by a rule-based bot, and despite human-inspired heuristics, they lack natural variability in timing, errors, and strategy. The dataset focuses solely on sketch-extrude workflows, omitting advanced operations like fillets, sweeps, and lofts. Interactions are limited to a single platform—OnShape—raising concerns about generalization. Finally, our current end-to-end benchmarks are only validated on a small subset due to the high cost of CAD rendering and geometric comparison. To address these gaps, future work will: (1) incorporate human demonstration data (e.g., from YouTube CAD tutorials), (2) extend coverage to assemblies and more advanced CAD features, (3) support additional CAD platforms (e.g., Fusion 360, FreeCAD), (4) collect multiple trajectories per CAD target to capture variation across users, (5) introduce extrusion-by-extrusion text prompts to enable natural interaction between users and AI agents in CAD softwares.

7 Conclusion

In this work, we introduced VIDEOCAD, a dataset for evaluating agents on utilizing complex computer software. Our dataset is significantly more complex than other datasets in number of action sequences and number of elements, as well as requiring agents to reason about 3D geometric spaces. VIDEOCAD extends beyond imitation learning and 3D reasoning to serve as a versatile benchmark across machine learning subfields. Its long, fully-observed action sequences support reinforcement learning and planning methods. The alignment between video, symbolic actions, and 3D outputs enables large-scale multimodal pre-training, which can be fine-tuned for tasks like action prediction, video segmentation, and shape retrieval. Its structured, geometry-changing workflows also make it valuable for research in computer vision (e.g., goal inference), HCI (e.g., tutorial generation), robotics (e.g., skill learning), and NLP (e.g., grounding language in actions). VIDEOCAD offers a foundation for developing generalist agents that can perceive, act, and reason in complex software environments.

References

- [1] Les A Piegl. Ten challenges in computer-aided design. *Computer-aided design*, 37(4):461–470, 2005.
- [2] Tianlin Shi, Andrej Karpathy, Linxi Fan, Jonathan Hernandez, and Percy Liang. World of bits: An open-domain platform for web-based agents. In Doina Precup and Yee Whye Teh,

- editors, *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 3135–3144. PMLR, 06–11 Aug 2017.
- [3] Biplab Deka, Zifeng Huang, Chad Franzen, Joshua Hibschan, Daniel Afegan, Yang Li, Jeffrey Nichols, and Ranjitha Kumar. Rico: A mobile app dataset for building data-driven design applications. In *Proceedings of the 30th Annual Symposium on User Interface Software and Technology*, UIST ’17, 2017.
 - [4] Rundi Wu, Chang Xiao, and Changxi Zheng. Deepcad: A deep generative network for computer-aided design models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 6772–6782, 2021.
 - [5] PTC Inc. Onshape. <https://www.onshape.com>.
 - [6] Evan Zheran Liu, Kelvin Guu, Panupong Pasupat, Tianlin Shi, and Percy Liang. Reinforcement learning on web interfaces using workflow-guided exploration. In *International Conference on Learning Representations (ICLR)*, 2018.
 - [7] Christopher Rawles, Alice Li, Daniel Rodriguez, Oriana Riva, and Timothy Lillicrap. Android in the wild: A large-scale dataset for android device control, 2023.
 - [8] Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents, 2023.
 - [9] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion, 2024.
 - [10] Bowen Baker, Ilge Akkaya, Peter Zhokhov, Joost Huizinga, Jie Tang, Adrien Ecoffet, Brandon Houghton, Raul Sampedro, and Jeff Clune. Video pretraining (vpt): Learning to act by watching unlabeled online videos, 2022.
 - [11] Zecheng He, Srinivas Sunkara, Xiaoxue Zang, Ying Xu, Lijuan Liu, Nevan Wichers, Gabriel Schubiner, Ruby Lee, Jindong Chen, and Blaise Agüera y Arcas. Actionbert: Leveraging user actions for semantic understanding of user interfaces, 2021.
 - [12] Difei Gao, Lei Ji, Zechen Bai, Mingyu Ouyang, Peiran Li, Dongxing Mao, Qinchen Wu, Weichen Zhang, Peiyi Wang, Xiangwu Guo, Hengxu Wang, Luowei Zhou, and Mike Zheng Shou. Assistgui: Task-oriented desktop graphical user interface automation, 2024.
 - [13] Sebastian Koch, Albert Matveev, Zhongshi Jiang, Francis Williams, Alexey Artemov, Evgeny Burnaev, Marc Alexa, Denis Zorin, and Daniele Panozzo. Abc: A big cad model dataset for geometric deep learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 9601–9611, 2019.
 - [14] Karl D.D. Willis, Yewen Pu, Jieliang Luo, Hang Chu, Tao Du, Joseph G. Lambourne, Armando Solar-Lezama, and Wojciech Matusik. Fusion 360 gallery: A dataset and environment for programmatic cad construction from human design sequences. *ACM Transactions on Graphics*, 40(4):54:1–54:24, 2021.
 - [15] Weijuan Cao, Trevor T. Robinson, Yang Hua, Flavien Boussuge, Andrew R. Colligan, and Wanbin Pan. Graph representation of 3d cad models for machining feature recognition with deep learning. In *ASME International Design Engineering Technical Conferences and Computers and Information in Engineering Conference*, 2020.
 - [16] Andrew R. Colligan, Trevor T. Robinson, Declan C. Nolan, Yang Hua, and Weijuan Cao. Hierarchical cadnet: Learning from b-reps for machining feature recognition. *Computer-Aided Design*, 147:103226, 2022.
 - [17] R. Kenny Jones, Theresa Barton, Xianghao Xu, Kai Wang, Ellen Jiang, Paul Guerrero, Niloy J. Mitra, and Daniel Ritchie. Shapeassembly: learning to generate programs for 3d shape structure synthesis. *ACM Transactions on Graphics*, 39(6):1–20, November 2020.

- [18] Mohammad Sadil Khan, Sankalp Sinha, Talha Uddin Sheikh, Didier Stricker, Sk Aziz Ali, and Muhammad Zeshan Afzal. Text2cad: Generating sequential cad models from beginner-to-expert level text prompts, 2024.
- [19] Karl D. D. Willis, Pradeep Kumar Jayaraman, Hang Chu, Yunsheng Tian, Yifei Li, Daniele Grandi, Aditya Sanghi, Linh Tran, Joseph G. Lambourne, Armando Solar-Lezama, and Wojciech Matusik. Joinable: Learning bottom-up assembly of parametric cad joints, 2022.
- [20] Negar Heidari and Alexandros Iosifidis. Geometric deep learning for computer-aided design: A survey, 2024.
- [21] Ari Seff, Yaniv Ovadia, Wenda Zhou, and Ryan P. Adams. Sketchgraphs: A large-scale dataset for modeling relational geometry in computer-aided design, 2020.
- [22] Wamiq Reyaz Para, Shariq Farooq Bhat, Paul Guerrero, Tom Kelly, Niloy Mitra, Leonidas Guibas, and Peter Wonka. Sketchgen: Generating constrained cad sketches, 2021.
- [23] Md Ferdous Alam and Faez Ahmed. Gencad: Image-conditioned computer-aided design generation with transformer-based contrastive representation and diffusion priors. *arXiv preprint arXiv:2409.16294*, 2024.
- [24] Pradeep Kumar Jayaraman, Joseph G. Lambourne, Nishkrit Desai, Karl D. D. Willis, Aditya Sanghi, and Nigel J. W. Morris. Solidgen: An autoregressive model for direct b-rep synthesis, 2023.
- [25] Jeffrey Buckley, Niall Seery, and Donal Canty. Heuristics and cad modelling: An examination of student behaviour during problem solving episodes within cad modelling activities. *International Journal of Technology and Design Education*, 29(4):939–956, 2018.
- [26] Peter Shaw, Mandar Joshi, James Cohan, Jonathan Berant, Panupong Pasupat, Hexiang Hu, Urvashi Khandelwal, Kenton Lee, and Kristina Toutanova. From pixels to ui actions: Learning to follow instructions via graphical user interfaces, 2023.
- [27] Tim Pearce and Jun Zhu. Counter-strike deathmatch with large-scale behavioural cloning, 2021.
- [28] OpenAI. Introducing gpt-4.1 in the api, April 2025.
- [29] Anthropic. Claude 3.7 sonnet and claude code, February 2025.
- [30] Qwen Team. Qwen2.5-vl technical report. *arXiv preprint arXiv:2502.13923*, 2025.
- [31] OpenAI. Introducing openai o3 and o4-mini, April 2025.
- [32] Google. Gemini 2.5 pro | generative ai on vertex ai, May 2025.
- [33] Thibault Le Sellier De Chezelles, Maxime Gasse, Alexandre Drouin, Massimo Caccia, Léo Boisvert, Megh Thakkar, Tom Marty, Rim Assouel, Sahar Omid Shahyeghan, Lawrence Keunho Jang, Xing Han Lù, Ori Yoran, Dehan Kong, Frank F. Xu, Siva Reddy, Quentin Cappart, Graham Neubig, Ruslan Salakhutdinov, Nicolas Chapados, and Alexandre Lacoste. The browsergym ecosystem for web agent research, 2025.
- [34] Yaobo Liang, Chenfei Wu, Ting Song, Wenshan Wu, Yan Xia, Yu Liu, Yang Ou, Shuai Lu, Lei Ji, Shaoguang Mao, et al. Taskmatrix. ai: Completing tasks by connecting foundation models with millions of apis. *Intelligent Computing*, 3:0063, 2024.
- [35] Lvmin Zhang, Anyi Rao, and Maneesh Agrawala. Adding conditional control to text-to-image diffusion models, 2023.

A Onshape

We use Onshape [5], a cloud-native CAD platform, as the environment for our task of learning CAD UI for creating and editing 3D models of mechanical parts and assemblies. Note that unlike traditional desktop-based CAD software, Onshape runs entirely in the browser and supports real-time collaboration, version control (similar to Git), and parametric design. We chose OnShape as it is free, platform agnostic (a majority of CAD software run on Windows), and highly accessible via the browser. In addition, OnShape contains most of the CAD operations used in standard commercial CAD software. We choose to use Google Chrome as the browser of choice as it is available on all main operating systems.

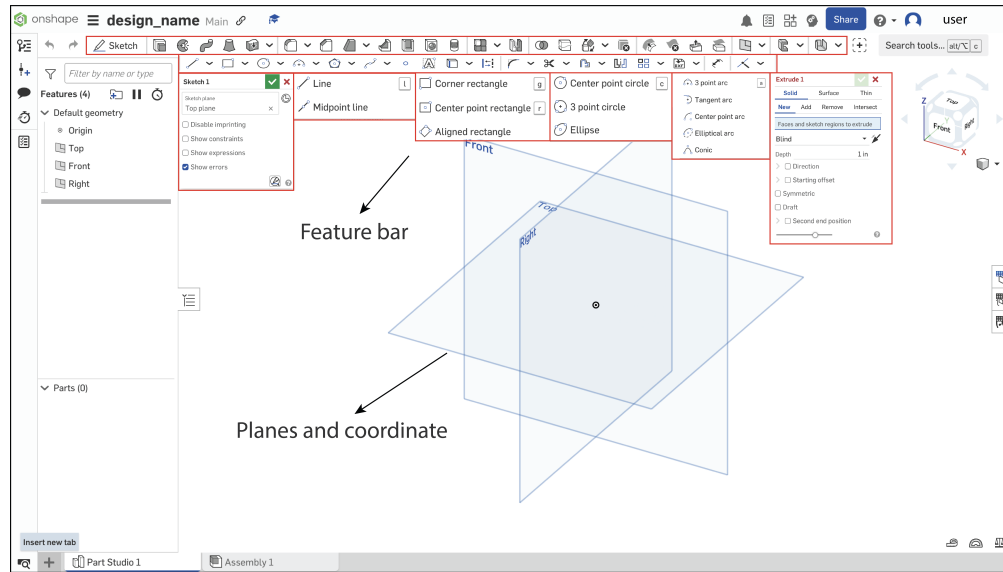


Figure 8: Overview of the Onshape user interface. The feature bar at the top includes general modeling operations as well as sketch-specific tools such as lines, rectangles, and circles. The left panel displays the feature tree with the default geometry (Top, Front, Right planes), while the central view shows the coordinate system and sketching planes. The right panel demonstrates options for the Extrude feature, used after defining sketches.

A.1 DeepCAD Representation

The DeepCAD sequence [4] represents modeling operations as a sequence of the form:

$$[t_i, x, y, \alpha, f, r, \theta, \phi, \gamma, p_x, p_y, p_z, s, e_1, e_2, u, b]$$

Each segment corresponds to a command: - t_i : command type (0: line, 1: arc, 2: circle, 4: loop separator, 5: extrusion) - x, y : points - α, f : arc angle and direction - r : radius (for circle) - θ, ϕ, γ : plane angles - p_x, p_y, p_z : sketch origin offset - s : sketch scale - e_1, e_2 : extrusion extents - u, b : extrusion operation type and symmetry

A.2 Normalization

All values from the DeepCAD sequences are in $[0, 255]$ and are normalized using:

$$\mathcal{N}(p) = \frac{p - 128}{128}$$

Used for coordinates, angles, plane offsets, and extrusion parameters.

A.3 Plane Basis and Extrusion Plane Parameters

The sketch plane is defined by normalized angles (θ, ϕ, γ) and a 3D offset (p_x, p_y, p_z) . Using:

$$(\theta, \phi, \gamma) = \pi \cdot \mathcal{N}([\theta, \phi, \gamma]), \quad \mathbf{o} = \mathcal{N}([p_x, p_y, p_z])$$

we compute an orthonormal basis $(\mathbf{n}, \mathbf{x}, \mathbf{y})$, where:

$$\mathbf{n} = \text{polar-to-cartesian}(\theta, \phi, \gamma), \quad \mathbf{y} = \mathbf{n} \times \mathbf{x}$$

The sketch plane is mapped to an integer ID:

$$\text{plane_id} \in \{0 : \text{Right}, 1 : \text{Front}, 2 : \text{Top}\}$$

The final extrusion plane offset is:

$$\text{offset} = 0.5 \cdot \mathbf{o}[\text{plane_id}]$$

A.4 Point Transformation to Pixel Space

Every geometric point in a sketch (e.g., endpoints, centers, midpoints) is transformed into 2D pixel coordinates through the following steps:

1. Normalization:

$$\mathbf{p}_{\text{norm}} = \mathcal{N}([x, y]) = \frac{[x, y] - 128}{128}$$

2. Rotation and Offset:

$$\mathbf{p}_{\text{rot}} = R \cdot \mathbf{p}_{\text{norm}} \cdot s + \mathbf{o}$$

where $R = [\mathbf{x}, \mathbf{y}]^T$ is the rotation matrix derived from the sketch plane basis, s is the global scale, and $\mathbf{o}$ is the origin offset.

3. Projection:

We project the rotated 3D point to 2D by dropping the axis corresponding to the sketch plane's normal direction.

Define a binary vector $\text{mask} \in \{0, 1\}^3$ that keeps the two in-plane components:

$$\text{mask}[i] = \begin{cases} 0 & \text{if } i = \text{plane_id} \\ 1 & \text{otherwise} \end{cases}$$

The 2D projected point becomes:

$$\mathbf{p}_{\text{proj}} = \mathbf{p}_{\text{rot}}[\text{mask}]$$

4. Pixel Alignment:

$$\mathbf{p}_{\text{pixel}} = 0.5 \cdot \mathbf{p}_{\text{proj}} + \mathbf{C}$$

where $\mathbf{C}$ is the pixel-space origin of the Onshape canvas.

This transformation is consistently applied to all sketch entities including:

- Line endpoints
- Circle centers
- Arc midpoints, center, start and end points

A.5 Primitive-Specific Parameter Computation

Each loop in the sketch is composed of geometric primitives: **lines**, **circles**, and **arcs**, each defined by a set of transformed parameters. Below, we define each primitive mathematically using pixel-space coordinates.

A.5.1 Lines

A line segment is defined by its start and end points. Let $\mathbf{p}_{\text{start}}, \mathbf{p}_{\text{end}} \in \mathbb{R}^2$ be the 2D pixel coordinates after projection:

$$\text{Line} = \{\mathbf{p}_{\text{start}}, \mathbf{p}_{\text{end}}\}$$

Each point is obtained by:

$$\mathbf{p} = 0.5 \cdot (R \cdot (\mathcal{N}([x, y]) \cdot s) + \mathbf{o}) [\text{mask}] + \mathbf{C}$$

A.5.2 Circles

A circle is defined by its center and radius. Let $\mathbf{c} \in \mathbb{R}^2$ be the projected center and r the scaled radius:

$$r = \frac{r_{\text{seq}}}{128} \cdot s \cdot 0.5$$

$$\text{Circle} = \{\mathbf{c}, r\}, \quad \text{with } \mathbf{c} = 0.5 \cdot (R \cdot (\mathcal{N}([x, y]) \cdot s) + \mathbf{o}) [\text{mask}] + \mathbf{C}$$

A.5.3 Arcs

An arc is defined by its start point $\mathbf{p}_{\text{start}}$, end point $\mathbf{p}_{\text{end}}$, center $\mathbf{c}_{\text{arc}}$, midpoint $\mathbf{p}_{\text{mid}}$, and radius r .

Angle and Direction.

$$\alpha = 180 \cdot \frac{\alpha_{\text{seq}}}{128}, \quad f \in \{0, 1\}$$

Chord Geometry.

$$\mathbf{v} = \mathbf{p}_{\text{end}} - \mathbf{p}_{\text{start}}, \quad L = \|\mathbf{v}\|, \quad \mathbf{p}_{\text{chord}} = \frac{\mathbf{p}_{\text{start}} + \mathbf{p}_{\text{end}}}{2}$$

Radius and Offset.

$$r = \frac{L}{2 \sin(\alpha/2 \cdot \pi/180)}, \quad h = \sqrt{r^2 - (L/2)^2}$$

Center and Midpoint.

$$\mathbf{v}_{\perp} = \frac{[-v_y, v_x]}{\|\mathbf{v}\|}, \quad \mathbf{c}_{\text{arc}} = \begin{cases} \mathbf{p}_{\text{chord}} + h \cdot \mathbf{v}_{\perp}, & \text{if } f = 1 \\ \mathbf{p}_{\text{chord}} - h \cdot \mathbf{v}_{\perp}, & \text{if } f = 0 \end{cases}$$

The angular midpoint is computed as:

$$\theta_{\text{start}} = \text{atan2}(p_{\text{start},y} - c_y, p_{\text{start},x} - c_x)$$

$$\theta_{\text{mid}} = \theta_{\text{start}} \pm \frac{\alpha}{2} \cdot \frac{\pi}{180}$$

$$\mathbf{p}_{\text{mid}} = \mathbf{c}_{\text{arc}} + r \cdot \begin{bmatrix} \cos(\theta_{\text{mid}}) \\ \sin(\theta_{\text{mid}}) \end{bmatrix}$$

Representation.

$$\text{Arc} = \{\mathbf{p}_{\text{start}}, \mathbf{p}_{\text{end}}, \mathbf{c}_{\text{arc}}, \mathbf{p}_{\text{mid}}, r\}$$

All points are transformed to pixel coordinates using the transformation pipeline from Section ??.

A.6 Extrusion Parameters

Extrusion parameters are computed as:

$$u \in \{0 : \text{new}, 1 : \text{remove}, 2 : \text{union}\}, \quad b \in \{0 : \text{one-sided}, 1 : \text{symmetric}, 2 : \text{two-sided}\}$$
$$e_1 = 0.5 \cdot \mathcal{N}(e_1), \quad e_2 = 0.5 \cdot \mathcal{N}(e_2)$$

A.7 Final Representation

Each extrusion is encoded as:

$$\text{Extrusion} = \{\text{plane_id}, \text{offset}, u, e_1, e_2, b, \text{profile}\}$$

Where profile is a list of loops, and each loop is a list of geometric primitives: lines, arcs, and circles with parameters as computed above.

B Detailed Onshape UI Action Procedure

This appendix provides a thorough description of our automated CAD modeling procedure within Onshape, driven by our hybrid UI interaction framework based on rule-based commands. The procedure systematically translates structured CAD command sequences (defined in Appendix ??) into executable UI interactions within Onshape.

B.1 Plane Creation Procedure

For sketches requiring custom-defined planes (offset from default planes), we follow these UI steps:

1. Activate the plane creation tool by clicking the plane icon.
2. Select one of the default reference planes (Top, Front, Right).
3. Navigate to the offset text box via successive presses of the Tab key.
4. Enter the desired offset value numerically.
5. Click the offset direction arrow to define plane offset orientation.
6. Finalize plane creation by pressing Enter.

B.2 Sketch Creation and Loop Building Procedure

Each sketch comprises loops defined by geometric primitives: lines, arcs, and circles. These loops are drawn via the following UI actions:

General Sketch Setup:

1. Start a new sketch by pressing Shift+S.
2. Select the appropriate sketch plane (custom or default).

Drawing Geometric Primitives:

- **Line:** Press L, then click to specify start and end points.
- **Circle:** Press C, click to set the center point, move cursor outward to specify radius, then click to finalize.
- **Arc (3-point):** Press A, sequentially click to set start, end, and midpoint.

Constraint Management: To ensure geometric accuracy, constraints are toggled dynamically:

- Press `Shift` key down to deactivate constraints.
- Press `Shift` key up to activate constraints (primarily when connecting loop endpoints).

Command Completion:

- Exit current geometric command (line, arc, circle) using `Esc`.

B.3 Visibility and Navigation

Visibility and navigation are managed to maintain clarity and prevent interference from previously drawn elements:

- Toggle visibility of planes: `Shift+P`.
- Toggle visibility of sketches: `Shift+H`.
- Hide/show previously built parts: `Y/Shift+Y`.
- Navigate between default planes (Top, Front, Right) using key sequences: `Shift (down) → + → (Up/Right/Down/Left) → Shift (up)`.

B.4 Extrusion Execution Procedure

Extrusions convert sketch loops into 3D features through these UI steps:

1. Select sketch region(s) intended for extrusion.
2. Activate extrusion using `Shift+E`.
3. Choose extrusion type (New for adding material, Remove for material removal).
4. Navigate (using `Tab`) to set the extrusion depth numerically.
5. Navigate further (using `Tab`) to the symmetric option checkbox; press `Space` to toggle symmetry on/off.
6. If performing material removal (Remove), navigate to and activate the “Merge with all” checkbox.
7. Finalize extrusion by pressing `Enter`.

B.5 End-of-Sequence Token

We use the hotkey combination `Shift+7` to set the camera view to an isometric perspective. This action signifies the completion of the CAD modeling sequence and serves as our end-of-sequence token.

C Ablation on Vision Models for CAD Image Similarity

Table 6: Comparison of CLIP and DINOv2 for CAD image similarity over 400 samples

Metric	CLIP	DINOv2
Mean of correct scores	0.8576	0.7923
Median of correct scores	0.8587	0.8022
Std Dev of correct scores	0.0250	0.0488
Mean of incorrect scores	0.7931	0.6002
Median of incorrect scores	0.7947	0.6014
Std Dev of incorrect scores	0.0167	0.0440
Average rank of correct match	18.24	2.21
Median rank of correct match	6.00	1.00
Std Dev of rank	31.69	3.08

We conduct an ablation comparing **DINOv2** and **CLIP** as image encoders for CAD shape comparison. For each of 400 query samples, we compute cosine similarity between features extracted from a rendered UI isometric image and a set of 400 ground-truth isometric CAD images (including the correct one). We define the *correct score* as the similarity between the UI image and its associated CAD image, and the *incorrect scores* as the similarities with all other CAD images in the set. The rank of the correct match is computed by ranking all 400 similarity scores in descending order.

Table 6 reports the results. Although CLIP yields slightly higher correct match scores on average, its incorrect scores are also high—indicating limited discriminative power for geometric retrieval. In contrast, DINOv2 achieves significantly lower similarity scores for incorrect matches (mean of 0.6002 vs. 0.7931 for CLIP), resulting in more robust separation and better retrieval accuracy. This is reflected in the rank-based metrics: DINOv2 achieves an average correct match rank of **2.21** and a median of **1.00**, far outperforming CLIP (18.24 and 6.00, respectively), with far less variance (std dev of 3.08 vs. 31.69).

This improvement can be attributed to the architectural and training differences between the models: **CLIP** is optimized for semantic alignment between text and image—learning “what is this”—whereas **DINOv2** is trained in a self-supervised manner to model fine-grained visual structure, capturing both local and global geometry. This makes DINOv2 naturally better suited for comparing CAD-like images, where shape and spatial configuration are more critical than semantic labels.

Threshold Selection. We also use these similarity scores to define a threshold for dataset quality control. Specifically, we set the threshold at 0.7, which corresponds to the average of the median correct score (0.8022) and median incorrect score (0.6014). This ensures a conservative yet robust filter for identifying high-quality image-CAD pairs.

D Dataset Benchmarking

The description of the metrics used to compare dataset complexity is explained in the following: **# Samples:** number of samples in the dataset, **Time Horizon:** number of UI interactions needed to complete the task, **Requires 3D Reasoning:** we look through every app used in each dataset. We determine whether the app requires the agent to reason about 3D coordinates to manipulate UI-state. **Contains Precise Element:** many benchmarks ground agents by leveraging the Document Object Model (DOM), enabling agents to select UI elements without needing to specify xy coordinates. However, some elements such as canvas elements cannot be interacted via the DOM. We define a precise element as an element that requires an agent to manipulate via xy coordinates, which require a higher level of reasoning as agents must rely on spatial and visual reasoning skills to click on the correct buttons. To check whether a benchmark requires a precise element, we look through every app used in each dataset and determine whether the agent is required to interact with a precise element such as a canvas, **Average UI elements:** To show that OnShape’s complexity as a GUI, we measure

the number of elements as described in Mind2Web. We only report the average number of elements for datasets that include the HTML tree in the dataset.

E Training Details and Ablation Studies

Training Details For VPT, we feed the UI Images into the CNN layer proposed by the paper, and then concatenate the output representation, the CAD image representation and the past actions and feed them into a feed forward network to get the command and parameters. For Pix2Act, we concatenate the CAD image and the past 10 images into a single image and feed it into the model, we then take the final hidden representation from the decoder and feed them into two linear layers that predict command and action parameters. For Pearce et. al., we do not modify the architecture.

All models were trained using 4 NVIDIA H100 GPUs. VPT and VideoCAD required 4 days of training, while Pix2Act and Pearce et al. converged in 1 day. Quantitative results are summarized in Table 7.

We train VIDEOCADFORMER on the VIDEOCAD dataset with a 90/5/5 train/val/test split. At each step, the model autoregressively predicts the next action given a window of $k = 10$ past UI frames and actions. Outputs are treated as classification tasks and trained using cross-entropy loss with inverse-frequency class weighting. For pointer and typed parameters (x, y, v), we apply soft targets over a ± 2 bin window to account for geometric tolerance.

Ablation Studies To investigate the contribution of various modeling components and hyperparameters, we perform an extensive ablation study covering 16 model variants. These ablations isolate specific architectural and input conditioning factors that influence performance in long-horizon CAD action prediction. For these models, we train our ablations on a 90/5/5 split on 2500 samples of data.

First, we vary the model capacity, examining small versus large hidden dimensions (hidden size $\in \{512, 1024, 2048\}$) and feedforward projection sizes (feed forward $\in \{512, 1024, 2048\}$). Results show that increasing hidden size improves performance.

Next, we assess the temporal window and attention configuration. Reducing or increasing the attention window (window size $\in \{5, 10, 15\}$) and varying transformer depth and heads (nhead $\in \{4, 8, 16\}$) allows us to probe the role of long-range context. Similar to model capacity, we observe increasing gains as we increase model capacity.

We also analyze the impact of input conditioning, disabling one or more context signals: past actions, past states, and timestep embeddings. Removing either past states or past actions significantly degrades performance, underscoring the importance of sequence memory in CAD modeling. The worst-performing variant disables both, indicating that VIDEOCADFORMER’s success hinges on leveraging temporal continuity.

Additionally, we introduce a color jitter variant to test robustness to visual perturbations. Adding jitter slightly degrades performance, suggesting some sensitivity to pixel-space distortions and motivating future robustness strategies.

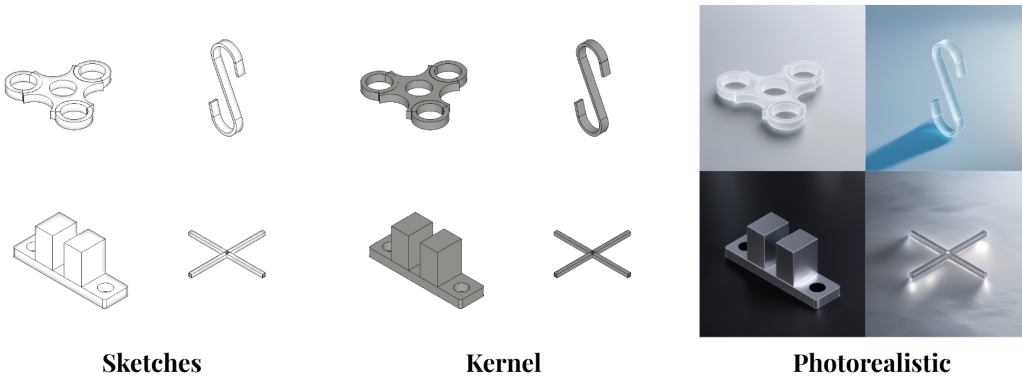


Figure 9: Comparison of different CAD image inputs. From left to right: Sketches, Kernel and Realistic

Table 7: Ablation results showing the impact of input types, image modalities, and architecture choices on command accuracy, parameter accuracy, and perfect action prediction across task difficulties

	μ_{cmd} (%)	μ_{param} (%)	Perfectly Predicted Actions (%)					
			Mean	Max	Min	Short	Medium	Long
Base model	94.90	71.55	79.27	97.21	54.24	83.88	76.94	76.03
Model Inputs								
No action	88.92	65.11	73.11	96.59	40.34	81.15	68.07	68.52
No state	94.34	68.07	77.86	97.73	56.36	82.90	75.76	73.85
No action and state	17.35	3.80	5.59	25.52	0.00	2.30	7.78	7.32
No timestep	95.03	70.97	79.39	95.81	57.58	83.85	77.52	75.85
Color jitter	94.82	71.03	79.24	96.65	51.53	84.03	77.00	75.68
Images								
Base model (kernel-generated image)	94.90	71.55	79.27	97.21	54.24	83.88	76.94	76.03
Realistic Images	94.05	68.50	77.62	95.45	56.57	83.20	74.86	73.64
Sketches	95.04	71.32	79.31	98.32	59.39	83.44	77.44	76.20
Window Size								
Past 5 actions and states	94.97	72.22	79.86	97.73	58.64	84.49	77.78	76.35
Base model (Past 10 actions and states)	94.90	71.55	79.27	97.21	54.24	83.88	76.94	76.03
Past 15 actions and states	94.17	69.10	77.88	95.45	54.24	82.90	75.72	73.96
Feed Forward (FF.) Dim								
FF. dim=512	94.63	70.38	79.08	96.59	55.93	84.04	76.80	75.36
Base model (FF. dim=1024)	94.90	71.55	79.27	97.21	54.24	83.88	76.94	76.03
FF. dim=2048	95.04	71.56	79.82	96.97	60.68	84.84	77.35	76.23
Hidden Size								
Hidden size=512	94.32	70.02	78.56	97.73	57.97	83.63	76.02	74.98
Base model (Hidden size = 1024)	94.90	71.55	79.27	97.21	54.24	83.88	76.94	76.03
Hidden size=2048	95.11	71.49	79.67	96.79	58.38	83.85	77.74	76.55
Nhead								
Nhead=4	94.54	70.06	78.84	96.37	57.29	83.56	76.99	74.96
Base model (Nhead = 8)	94.90	71.55	79.27	97.21	54.24	83.88	76.94	76.03
Nhead=16	95.70	72.19	79.96	98.04	62.37	84.43	77.51	77.03

Finally, we explore VIDEOCADFORMER’s ability to generalize to different image types such as realistic images and sketches (Figure 9). We first generated images from a DeepCAD sequence using a geometry kernel. From this kernel-generated image, we constructed photo-realistic images by passing the kernel-generated CAD images into ControlNet [35]. The sketches are generated by applying canny line detection to the kernel-generated CAD images and then applying a Gaussian blur. Among the three, the model performs best on sketches, followed by kernel-generated images, and performs worst on photorealistic images, suggesting increased abstraction aids generalization.

We train a baseline model to act as a point of reference. The architectural parameters of the baseline model are shown in the table. Additionally, the baseline model has access to all inputs and are trained on grayscale isometric views of CAD images.

Overall, these ablations demonstrate that model performance is heavily influenced by temporal conditioning, architecture depth, and memory of past actions/states. These insights can guide the design of future agents operating in long-horizon, precision-driven software environments.

F Metrics

Chamfer Distance after PCA Alignment. To quantitatively evaluate the geometric similarity between generated and ground truth CAD models, we begin by sampling point clouds uniformly from the surface meshes of each model. Let $P \subset \mathbb{R}^3$ denote the ground truth point cloud and $\hat{P} \subset \mathbb{R}^3$ denote the point cloud sampled from the generated CAD model. Because these models may differ in scale, orientation, or position, we first apply a similarity transformation to $\hat{P}$ that aligns it with P before computing the metric.

This transformation consists of a rotation $R \in \text{SO}(3)$, uniform scale $s \in \mathbb{R}$, and translation $t \in \mathbb{R}^3$, obtained via principal component analysis (PCA) alignment and RMS-based scale matching. Among all 48 possible PCA axis permutations and sign flips, we select the transformation that minimizes the Chamfer Distance between the aligned prediction and the ground truth.

Formally, we define the aligned predicted point cloud as:

$$\hat{P}_{\text{aligned}} = \left\{ sR\hat{p} + t \mid \hat{p} \in \hat{P} \right\},$$

and compute the symmetric Chamfer Distance as:

$$\text{CD}(P, \hat{P}_{\text{aligned}}) = \frac{1}{|P|} \sum_{p \in P} \min_{\hat{p} \in \hat{P}_{\text{aligned}}} \|p - \hat{p}\|^2 + \frac{1}{|\hat{P}_{\text{aligned}}|} \sum_{\hat{p} \in \hat{P}_{\text{aligned}}} \min_{p \in P} \|\hat{p} - p\|^2.$$

We report the mean Chamfer Distance after this alignment as a measure of geometric fidelity.

Command and Parameter Accuracy.

$$\mu_{\text{cmd}} = \frac{1}{T} \sum_{t=1}^T \mathbb{1}[\hat{c}_t = c_t], \quad \mu_{\text{param}} = \frac{1}{T} \sum_{t=1}^T \frac{1}{d_t} \sum_{i=1}^{d_t} \mathbb{1}[\hat{p}_t^{(i)} = p_t^{(i)}] \cdot \mathbb{1}[\hat{c}_t = c_t]$$

ambiguous in the rendered images. These issues highlight the difficulty of resolving fine-grained geometric distinctions from image supervision alone and suggest that incorporating topological constraints or fine-tuning with interactive feedback could further improve model robustness.

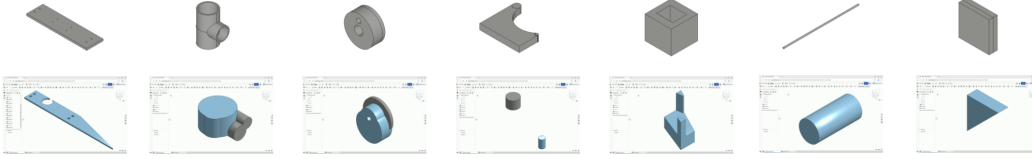


Figure 11: Failure cases

I Benchmarking LLMs’ 3D Reasoning Capabilities

To assess LLMs’ capabilities in spatial reasoning within CAD environments, we construct a synthetic Visual Question Answering (VQA) benchmark using the VIDEOCAD dataset. Each question is designed to probe specific aspects of geometric, temporal, and procedural reasoning essential for understanding 3D modeling workflows. The benchmark spans the following questions:

Table 8: Questions from our CAD-VQA dataset

Evaluation Type	Question Content	Choice Type
Extrusion Shape Prediction	You are given a completed sketch in OnShape. If the next command is Extrude, which image among the following will result from it?	Select correct image from 4 options
Num. Extrusion Estimation	How many extrusions were used in the provided CAD image?	Multiple choice (integers)
Extrusion Difference Prediction	You are given two extrusions for the same CAD model and the image of the CAD model. The second extrusion happens later than the first. Is the second extrusion deeper than the first?	Binary (Yes/No)
Sketch Ordering	Given these sketches from the video, order them to build the CAD object.	Identify correct sequence of sketches
Sketch Identification	Given an isometric view of a CAD model, select a sketch that was used to build this shape.	Select correct image from 4 options
Plane Identification	Given the following sketch and CAD image, which plane are you currently looking at?	Multiple choice: Top / Front / Right
CAD Primitive Identification	Which frame best matches the description of a given CAD primitive (arc, line, circle, extrude)?	Select correct image from 3 options
Sequence Prediction	What is the next primitive to draw (e.g., line, arc, etc.) given this CAD image and UI image?	Categorical multiple choice
Video Frame Sequencing	You are given 3 frames from the same video. What is the order of the frames?	Permutation over 3 items (6 options)
Hole Detection	Given this CAD image, is there a hole?	Binary (Yes/No)
Symmetry Detection	You are given an image of a CAD model. Across which planes is this CAD model symmetric?	Permutation over 3 planes (x,y,z) for a total of 8 options

Collectively, these questions exercise geometric recognition, viewpoint awareness, forward simulation, temporal planning, and numeric inference — topics that are rarely interrogated simultaneously by existing VQA benchmarks. We generated 200 samples for each question for a total of 2,200 samples, though this question generation process can be easily scaled to the entire dataset. We provide the full benchmarking results on the questions above in Table 5.